

Chapter 4: File Systems

Questions and Answers

Question 1

Give five different path names for the file `/etc/passwd`. (Hint: Think about the directory entries “.” and “..”.)

Answer 1

The directory entries `.` and `..` can be used to create many equivalent absolute paths.

```
/etc/passwd
../etc/passwd
../../etc/passwd
/etc../etc/passwd
/etc../etc../etc/passwd
```

All of these resolve to the same file.

Question 2

In Windows, when a user double-clicks on a file listed by Windows Explorer, a program is run and given that file as a parameter. List two different ways the operating system could know which program to run.

Answer 2

One method is to use the file extension, such as `.docx` or `.pdf`, and look up the associated application. Another method is to store metadata recording which program created or owns the file, as classic Macintosh systems did.

Question 3

In early UNIX systems, executable files (`a.out` files) began with a very specific magic number, not one chosen at random. These files began with a header, followed by the text and data segments. Why do you think a very specific number was chosen for executable files, whereas other file types had a more-or-less random magic number as the first word-

Answer 3

The magic number was chosen so it could also act as an executable instruction, typically a branch over the header. This allowed the binary file to be loaded directly at address 0 and started without first parsing the header size.

Question 4

Is the open system call in UNIX absolutely essential- What would the consequences be of not having it-

Answer 4

It is not absolutely essential, but removing it would make file I/O clumsy. Every `read` or `write` would need to include the file name, and the OS would repeatedly locate the file's i-node. Caching could reduce the cost, but issues such as when to flush modified metadata would

become awkward.

Question 5

Systems that support sequential files always have an operation to rewind files. Do systems that support random-access files need this, too-

Answer 5

No. In a random-access file, rewinding is just seeking to byte 0.

Question 6

Some operating systems provide a system call `rename` to give a file a new name. Is there any difference at all between using this call to rename a file and just copying the file to a new file with the new name, followed by deleting the old one-

Answer 6

No. `rename` preserves file identity and metadata such as creation and modification times. Copying creates a new file with new timestamps and may fail if there is not enough free disk space.

Question 7

In some systems it is possible to map part of a file into memory. What restrictions must such systems impose- How is this partial mapping implemented-

Answer 7

The mapped region should start on a page boundary and have a length that is an integral number of pages. Each mapped page uses the file itself as backing store. Pages not backed by the mapped file use the normal swap area or scratch backing store.

Question 8

A simple operating system supports only a single directory but allows it to have arbitrarily many files with arbitrarily long file names. Can something approximating a hierarchical file system be simulated- How-

Answer 8

Yes. Encode the full path into the single file name, for example `/usr/ast/file`. It only looks hierarchical; internally it is one long name stored in the single directory.

Question 9

In UNIX and Windows, random access is done by having a special system call that moves the “current position” pointer associated with a file to a given byte in the file. Propose an alternative way to do random access without having this system call.

Answer 9

Add an offset parameter to each `read` or `write`. Then every operation specifies the file position directly.

Tradeoff

This removes **seek**, but every I/O call becomes more complicated, and the user program must manage positions explicitly.

Question 10

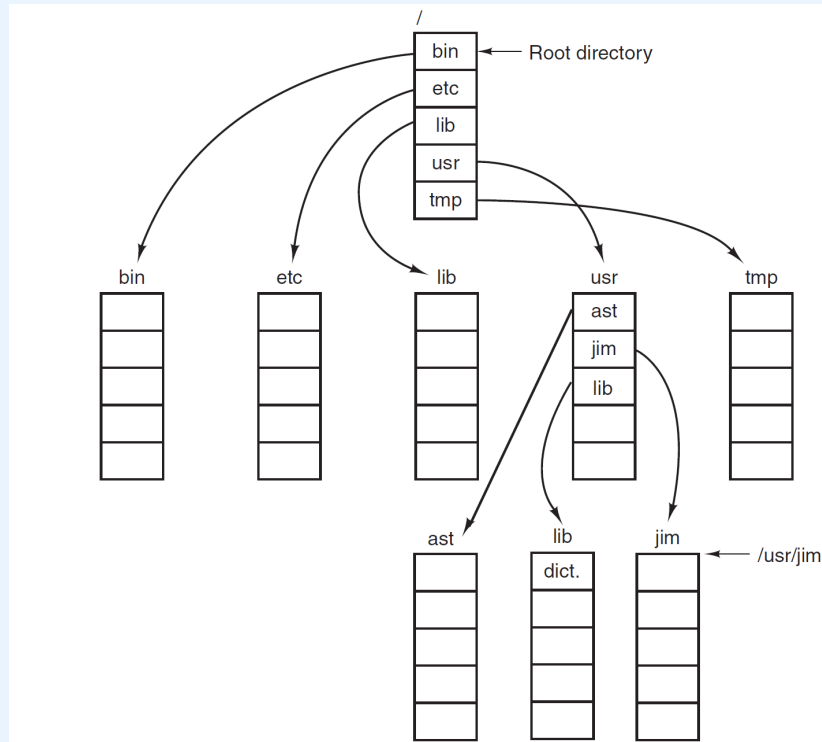


Figure 4-8. A UNIX directory tree.

Consider the directory tree of Fig. 4-8. If `/usr/jim` is the working directory, what is the absolute path name for the file whose relative path name is `../ast/x`?

Answer 10

`..` moves from `/usr/jim` to `/usr`. Then `ast/x` is resolved below `/usr`.

`/usr/ast/x`

Question 11

Contiguous allocation of files leads to disk fragmentation, as mentioned in the text, because some space in the last disk block will be wasted in files whose length is not an integral number of blocks. Is this internal fragmentation or external fragmentation- Make an analogy with something discussed in the previous chapter.

Answer 11

It is **external fragmentation**. The wasted space is between allocated files, not inside an allocated unit. This is analogous to external fragmentation in memory systems using swapping or pure segmentation.

Question 12

Describe the effects of a corrupted data block for a given file for: (a) contiguous, (b) linked, and (c) indexed (or table based).

Answer 12

Allocation	Effect of one corrupted data block
Contiguous	Only that block's data is damaged; the rest of the file can still be located.
Linked	The corrupted block's data is lost, and if its pointer is corrupted, all following blocks may become unreachable.
Indexed/table based	Only that data block is affected if the index/table remains correct.

Question 13

One way to use contiguous allocation of the disk and not suffer from holes is to compact the disk every time a file is removed. Since all files are contiguous, copying a file requires a seek and rotational delay to read the file, followed by the transfer at full speed. Writing the file back requires the same work. Assuming a seek time of 5 msec, a rotational delay of 4 msec, a transfer rate of 80 MB/sec, and an average file size of 8 KB, how long does it take to read a file into main memory and then write it back to the disk at a new location? Using these numbers, how long would it take to compact half of a 16-GB disk?

Answer 13

Starting a read costs:

$$5 + 4 = 9 \text{ msec}$$

Transfer time for 8 KB:

$$\frac{8192}{80 \times 2^{20}} \approx 0.0977 \text{ msec}$$

Read plus write:

$$2(9.0977) = 18.1954 \text{ msec per file}$$

Half of a 16-GB disk is 8 GB. With 8-KB files:

$$\frac{8 \text{ GB}}{8 \text{ KB}} = 2^{20} \text{ files}$$

$$2^{20} \times 18.1954 \text{ msec} \approx 19079 \text{ sec} \approx 5.3 \text{ hours}$$

about 5.3 hours

Question 14

In light of the answer to the previous question, does compacting the disk ever make any sense-

Answer 14

Yes, if done periodically and intelligently. During compaction, the system can place each file's blocks contiguously to improve future access. It is too expensive to do after every deletion, but periodic defragmentation can improve performance on fragmented disks.

Question 15

Some digital consumer devices need to store data, for example as files. Name a modern device that requires file storage and for which contiguous allocation would be a fine idea.

Answer 15

A digital camera is a good example. It writes photos sequentially to flash storage, and the storage is often emptied later when photos are transferred elsewhere. Contiguous allocation is simple and efficient for this workload.

Question 16

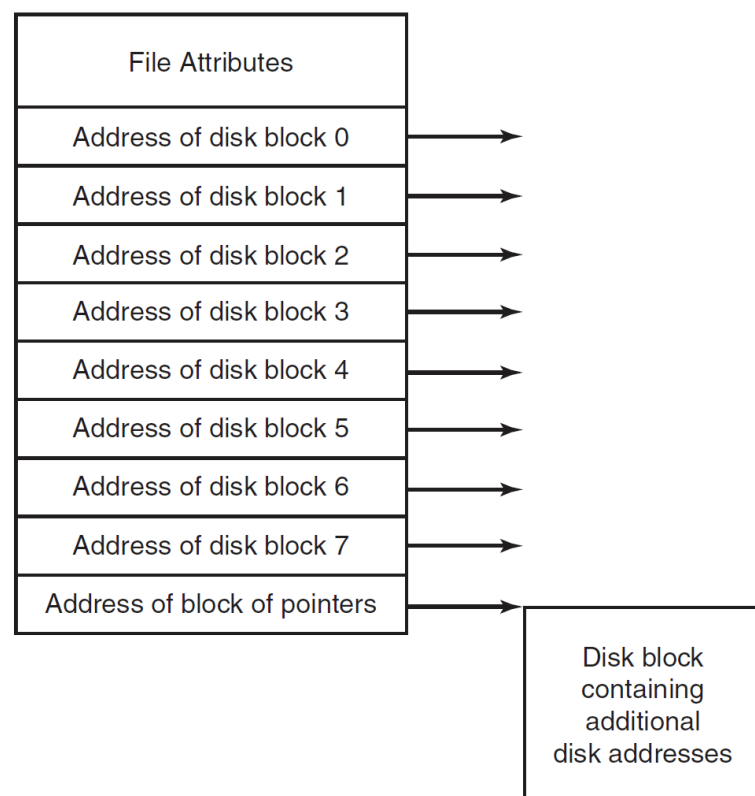


Figure 4-13. An example i-node.

Consider the i-node shown in Fig. 4-13. If it contains 10 direct addresses and these were 8 bytes each and all disk blocks were 1024 KB, what would the largest possible file be?

Answer 16

One indirect block holds:

$$\frac{1024}{8} = 128 \text{ addresses}$$

Total addressed blocks:

$$10 + 128 = 138$$

Largest file:

$$138 \times 1024 = 141,312 \text{ bytes}$$

138 KB

Question 17

For a given class, the student records are stored in a file. The records are randomly accessed and updated. Assume that each student's record is of fixed size. Which of the three allocation schemes (contiguous, linked and table/indexed) will be most appropriate-

Answer 17

Contiguous or indexed/table allocation is appropriate because both support random access efficiently. Linked allocation is poor because finding a random record may require following many links.

Question 18

Consider a file whose size varies between 4 KB and 4 MB during its lifetime. Which of the three allocation schemes (contiguous, linked and table/indexed) will be most appropriate-

Answer 18

Linked or indexed/table allocation is more appropriate than contiguous allocation. The large size variation makes contiguous allocation costly because the file may need relocation or compaction as it grows and shrinks. Indexed allocation is better than linked allocation when random access is important.

Question 19

It has been suggested that efficiency could be improved and disk space saved by storing the data of a short file within the i-node. For the i-node of Fig. 4-13, how many bytes of data could be stored inside the i-node-

Answer 19

The system needs a way to distinguish inline data from block pointers. If an unused attribute bit can mark inline data, all nine pointer fields can hold data:

9k bytes

where k is the size of one pointer field. If no bit is available, one pointer field must hold an invalid marker, leaving:

8k bytes

Question 20

Two computer science students, Carolyn and Elinor, are having a discussion about inodes. Carolyn maintains that memories have gotten so large and so cheap that when a file is opened, it is simpler and faster just to fetch a new copy of the i-node into the i-node table, rather than search the entire table to see if it is already there. Elinor disagrees. Who is right-

Answer 20

No. Elinor is right. Multiple in-memory copies of the same writable i-node can corrupt meta-data. If two copies are updated independently, the last one written back can overwrite changes made by the other.

Question 21

Name one advantage of hard links over symbolic links and one advantage of symbolic links over hard links.

Answer 21

Hard links need no extra file data block; they are just additional directory entries pointing to the same i-node. Symbolic links can point across file systems or even to files on remote machines, while hard links are usually limited to the same partition.

Question 22

Explain how hard links and soft links differ with respect to i-node allocations.

Answer 22

All hard links to a file point to the same i-node. A symbolic link has its own i-node and stores a pathname pointing to the target file.

Question 23

Consider a 4-TB disk that uses 4-KB blocks and the free-list method. How many block addresses can be stored in one block-

Answer 23

Number of disk blocks:

$$\frac{4 \text{ TB}}{4 \text{ KB}} = 2^{30}$$

A block number needs 30 bits, rounded up to 32 bits or 4 bytes. One 4-KB block holds:

$$\frac{4096}{4} = 1024$$

1024 block addresses

Question 24

Free disk space can be kept track of using a free list or a bitmap. Disk addresses require D bits. For a disk with B blocks, F of which are free, state the condition under which the free list uses less space than the bitmap. For D having the value 16 bits, express your answer as a percentage of the disk space that must be free.

Answer 24

Bitmap size is B bits. Free-list size is DF bits. The free list is smaller when:

$$DF < B$$

$$\frac{F}{B} < \frac{1}{D}$$

For $D = 16$:

$$\frac{F}{B} < \frac{1}{16} = 6.25\%$$

free list is smaller if less than about 6% of the disk is free

Question 25

The beginning of a free-space bitmap looks like this after the disk partition is first formatted: 1000 0000 0000 0000 (the first block is used by the root directory). The system always searches for free blocks starting at the lowest-numbered block, so after writing file A, which uses six blocks, the bitmap looks like this: 1111 1110 0000 0000. Show the bitmap after each of the following additional actions: (a) File B is written, using five blocks. (b) File A is deleted. (c) File C is written, using eight blocks. (d) File B is deleted.

Answer 25

Action	Bitmap
B written, five blocks	1111 1111 1111 0000
A deleted	1000 0001 1111 0000
C written, eight blocks	1111 1111 1111 1100
B deleted	1111 1110 0000 1100

Question 26

What would happen if the bitmap or free list containing the information about free disk blocks was completely lost due to a crash- Is there any way to recover from this disaster, or is it bye-bye disk- Discuss your answers for UNIX and the FAT-16 file system separately.

Answer 26

In UNIX, recovery is possible by scanning all i-nodes, listing every block used by every file, and taking the complement as the free list or bitmap. In FAT-16, the free-list problem does not arise in the same way because allocation information is in the FAT; free entries can be found by scanning the FAT.

Question 27

Oliver Owl's night job at the university computing center is to change the tapes used for overnight data backups. While waiting for each tape to complete, he works on writing his thesis that proves Shakespeare's plays were written by extraterrestrial visitors. His text processor runs on the system being backed up since that is the only one they have. Is there a problem with this arrangement-

Answer 27

Yes. A file open for writing may be in an inconsistent state during backup. The backup may skip it or copy a partial/inconsistent version, so his thesis may not be backed up reliably.

Question 28

We discussed making incremental dumps in some detail in the text. In Windows it is easy to tell when to dump a file because every file has an archive bit. This bit is missing in UNIX. How do UNIX backup programs know which files to dump-

Answer 28

They record the time of the last dump in a file. During the next backup, they dump files whose modification times are later than the last recorded dump time.

Question 29

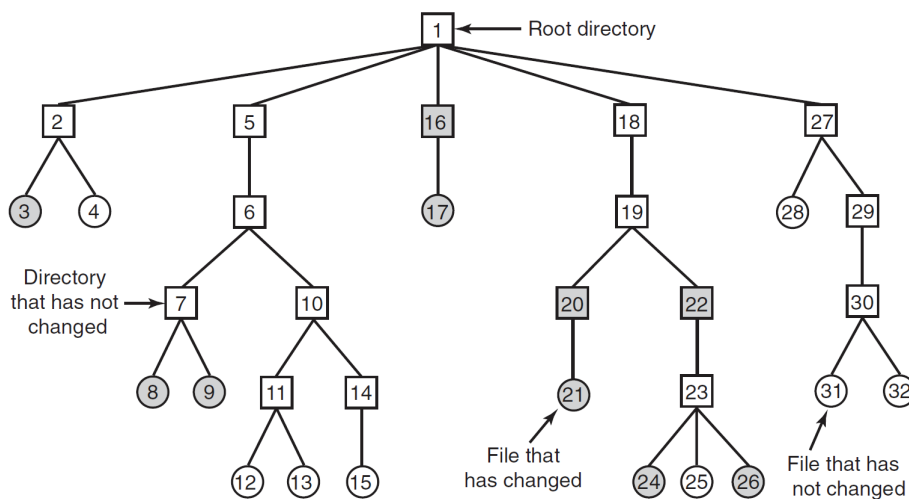


Figure 4-25. A file system to be dumped. The squares are directories and the circles are files. The shaded items have been modified since the last dump. Each directory and file is labeled by its i-node number.

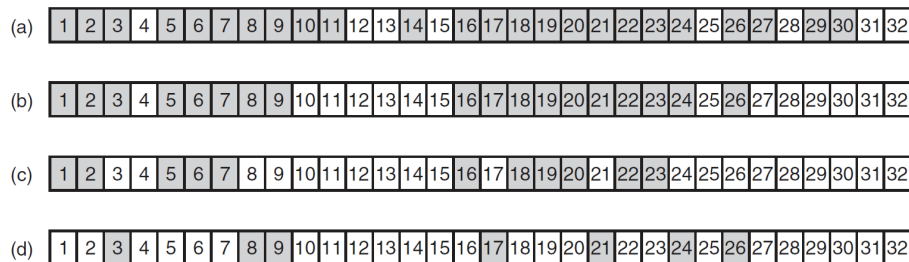


Figure 4-26. Bitmaps used by the logical dumping algorithm.

Suppose that file 21 in Fig. 4-25 was not modified since the last dump. In what way would the four bitmaps of Fig. 4-26 be different?

Answer 29

File 21 would not be marked in bitmaps (a) and (b). Bitmap (c) would be unchanged. File 21 would not be marked in bitmap (d).

Question 30

It has been suggested that the first part of each UNIX file be kept in the same disk block as its i-node. What good would this do-

Answer 30

Many files are short. If the file data fits with the i-node, the OS can read the i-node and the whole file in one disk access instead of two. Even for longer files, the first data block is obtained with one fewer disk access.

Question 31

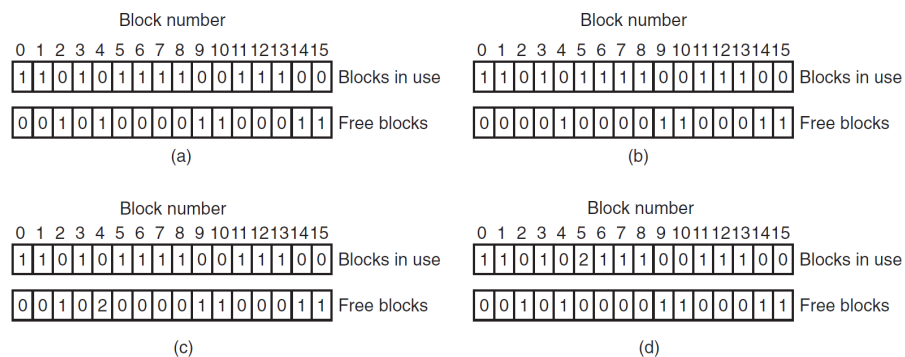


Figure 4-27. File-system states. (a) Consistent. (b) Missing block. (c) Duplicate block in free list. (d) Duplicate data block.

Consider Fig. 4-27. Is it possible that for some particular block number the counters in both lists have the value 2? How should this problem be corrected?

Answer 31

It should not happen, but a bug or crash could cause it. It means the block appears in two files and twice in the free list. Repair by removing both free-list copies, allocating a new free block, copying the bad shared block to it, and changing one file to point to the new copy.

Question 32

The performance of a file system depends upon the cache hit rate (fraction of blocks found in the cache). If it takes 1 msec to satisfy a request from the cache, but 40 msec to satisfy a request if a disk read is needed, give a formula for the mean time required to satisfy a request if the hit rate is h . Plot this function for values of h varying from 0 to 1.0.

Answer 32

$$T(h) = h(1) + (1 - h)(40)$$

$$T(h) = 40 - 39h \text{ msec}$$

The graph is a straight line from 40 msec at $h = 0$ to 1 msec at $h = 1$.

Question 33

For an external USB hard drive attached to a computer, which is more suitable: a write-through cache or a block cache-

Answer 33

A write-through cache is safer. It writes updates to the external drive immediately as well as updating the cache, reducing data loss if the user disconnects the drive before a delayed write-back occurs.

Question 34

Consider an application where students' records are stored in a file. The application takes a student ID as input and subsequently reads, updates, and writes the corresponding student record; this is repeated till the application quits. Would the "block readahead" technique be useful here-

Answer 34

Usually no. Read-ahead helps sequential access. Student IDs may be requested in arbitrary order, so the next block is not predictably the next sequential disk block.

Question 35

Consider a disk that has 10 data blocks starting from block 14 through 23. Let there be 2 files on the disk: f1 and f2. The directory structure lists that the first data blocks of f1 and f2 are respectively 22 and 16. Given the FAT table entries as below, what are the data blocks allotted to f1 and f2? (14,18); (15,17); (16,23); (17,21); (18,20); (19,15); (20,-1); (21, -1); (22,19); (23,14). In the above notation, (x, y) indicates that the value stored in table entry x points to data block y.

Answer 35

Follow the FAT chain until -1.

$$f1 : 22 \rightarrow 19 \rightarrow 15 \rightarrow 17 \rightarrow 21 \rightarrow -1$$

$$f2 : 16 \rightarrow 23 \rightarrow 14 \rightarrow 18 \rightarrow 20 \rightarrow -1$$

$$f1 = \{22, 19, 15, 17, 21\}, \quad f2 = \{16, 23, 14, 18, 20\}$$

Question 36

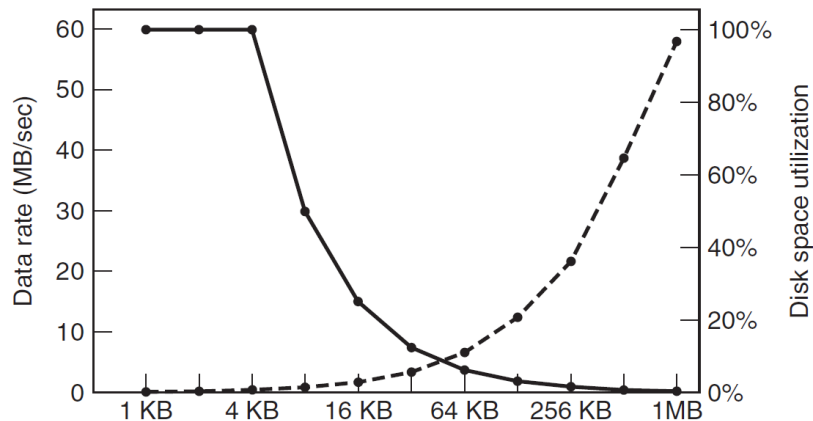


Figure 4-21. The dashed curve (left-hand scale) gives the data rate of a disk. The solid curve (right-hand scale) gives the disk-space efficiency. All files are 4 KB.

Consider the idea behind Fig. 4-21, but now for a disk with a mean seek time of 6 msec, a rotational rate of 15,000 rpm, and 1,048,576 bytes per track. What are the data rates for block sizes of 1 KB, 2 KB, and 4 KB, respectively?

Answer 36

At 15,000 rpm, one rotation takes 4 msec, so average rotational delay is 2 msec. For a block of k bytes:

$$T = 6 + 2 + \frac{k}{1,048,576} \times 4 \text{ msec}$$

Block	Time	Approx. rate
1 KB	8.0039 msec	128 KB/sec
2 KB	8.0078 msec	256 KB/sec
4 KB	8.0156 msec	511 KB/sec

Textbook Note

Some printed solutions report rates based on about 6 msec access time, but including the stated average rotational delay gives the values above.

Question 37

A certain file system uses 4-KB disk blocks. The median file size is 1 KB. If all files were exactly 1 KB, what fraction of the disk space would be wasted? Do you think the wastage for a real file system will be higher than this number or lower than it? Explain your answer.

Answer 37

Each 1-KB file consumes one 4-KB block, wasting 3 KB:

$$\frac{3}{4} = 75\%$$

75% wasted

Real wastage is usually lower because larger files use most of their blocks efficiently.

Question 38

Given a disk-block size of 4 KB and block-pointer address value of 4 bytes, what is the largest file size (in bytes) that can be accessed using 10 direct addresses and one indirect block?

Answer 38

One indirect block holds:

$$\frac{4096}{4} = 1024 \text{ pointers}$$

Total blocks:

$$10 + 1024 = 1034$$

Largest file:

$$1034 \times 4096 = 4,235,264 \text{ bytes}$$

4,235,264 bytes

Question 39

Write a program that reverses the bytes of a file, so that the last byte is now first and the first byte is now last. It must work with an arbitrarily long file, but try to make it reasonably efficient.

Answer 39

Use two fixed-size buffers, one reading from the front and one from the back. Swap chunks until the pointers meet. Reverse each chunk in memory before writing it to the opposite side.

```
left = 0
right = file_size
while left < right:
    read chunk from left
    read chunk from right - chunk_size
    reverse both chunks
    write former right chunk at left
    write former left chunk at right - chunk_size
    advance left; decrease right
```

This works for very large files because memory use is bounded by the buffer size.

Question 40

Write a program that starts at a given directory and descends the file tree from that point recording the sizes of all the files it finds. When it is all done, it should print a histogram of the file sizes using a bin width specified as a parameter (e.g., with 1024, file sizes of 0 to 1023

go in one bin, 1024 to 2047 go in the next bin, etc.).

Answer 40

Use recursive directory traversal. For each regular file, get its size using file metadata and compute:

$$\text{bin} = \left\lfloor \frac{\text{size}}{\text{bin width}} \right\rfloor$$

Increment that bin. After traversal, print each bin range and count. Skip or report files that cannot be accessed due to permissions.

Question 41

Write a program that scans all directories in a UNIX file system and finds and locates all i-nodes with a hard link count of two or more. For each such file, it lists together all file names that point to the file.

Answer 41

Traverse all directories and call `stat` or `lstat` on each entry. Use the pair (st_dev, st_ino) as the key and store all pathnames for that key. After scanning, print entries whose link count is at least 2 or whose collected name list has length at least 2.

Question 42

Write a new version of the UNIX `ls` program. This version takes as an argument one or more directory names and for each directory lists all the files in that directory, one line per file. Each field should be formatted in a reasonable way given its type. List only the first disk address, if any.

Answer 42

For each directory argument, call directory-reading APIs such as `opendir` and `readdir`. For each entry, call `stat` to obtain type, permissions, owner, size, timestamps, and block information. Format one line per file.

Implementation Note

Portable POSIX does not expose exact disk addresses directly; systems that need the first disk block may require filesystem-specific APIs or privileged tools.

Question 43

Implement a program to measure the impact of application-level buffer sizes on read time. This involves writing to and reading from a large file (say, 2 GB). Vary the application buffer size (say, from 64 bytes to 4 KB). Use timing measurement routines (such as `gettimeofday` and `getitimer` on UNIX) to measure the time taken for different buffer sizes. Analyze the results and report your findings: does buffer size make a difference to the overall write time and per-write time?

Answer 43

Create a large file, such as 2 GB, and read/write it repeatedly using buffer sizes from 64 bytes to 4 KB or larger. Measure elapsed time with `gettimeofday`, `clock_gettime`, or an equivalent timer.

Report total time and per-operation time:

$$\text{per-write time} = \frac{\text{total time}}{\text{number of writes}}$$

Small buffers usually perform worse because they cause many system calls. Larger buffers reduce system-call overhead and improve throughput until the benefit levels off.

Question 44

Implement a simulated file system that will be fully contained in a single regular file stored on the disk. This disk file will contain directories, i-nodes, free-block information, file data blocks, etc. Choose appropriate algorithms for maintaining free-block information and for allocating data blocks (contiguous, indexed, linked). Your program will accept system commands from the user to create/delete directories, create/delete/open files, read/write from/to a selected file, and to list directory contents.

Answer 44

Use the regular file as a virtual disk divided into fixed-size blocks. Store a superblock, free-space bitmap or free list, i-node table, directories, and data blocks inside it.

Support commands such as:

```
mkdir, rmdir, create, delete, open, close, read, write, ls
```

A simple design uses indexed allocation with i-nodes because it supports growth and random access better than contiguous allocation. The simulator should update metadata atomically where possible and include a consistency-check command for debugging.